

Notebook Checkpoint Walkthrough

Phases 4-7 -- Peer Discussion Guide + Q&A

Recap + Context

Quick recap of the first walkthrough. The business problem is CMS HRRP penalties -- hospitals lose up to 3% of Medicare payments for excess 30-day readmissions. We're working with the UCI Diabetes dataset: 102,000 encounters cleaned down to about 70,000 unique patients after deduplication and removing deceased patients.

Phases 1-3 covered data profiling, EDA with five statistical tests, and a complete sklearn pipeline with eight cleaning steps and seven engineered features. The key EDA findings were that prior inpatient visits, discharge disposition, and HbA1c measurement were the strongest signals. We set success targets before modeling: AUC above 0.65 and recall above 0.50.

The test set has been locked since Phase 3 -- 80/20 stratified split. Everything you're about to see in Phases 4 and 5 was done entirely through cross-validation on the training set. The test set only gets unlocked once, in Phase 6.

Phase 4: Baselines

Scroll to the utility functions cell

Before training any models, I defined three utility functions that get reused across Phases 4, 5, and 6. This was based on peer feedback from the first walkthrough -- wrapping repeated processes into functions.

- `evaluate_model` prints a full metrics report and confusion matrix.
- `predict_at_threshold` converts probabilities to binary predictions at any cutoff, not just 0.5.
- `compute_business_impact` simulates the dollar value of deploying the model at a given threshold.

These save about 50 lines of duplicated code across the remaining phases.

Scroll to the model definitions cell

Four models: Logistic Regression, Random Forest, XGBoost, and SVM. All four use `class_weight='balanced'` or the equivalent -- XGBoost uses `scale_pos_weight`, which is the ratio of negative to positive samples, about 13 to 1. Every model is wrapped in a Pipeline with the preprocessor from Phase 3, so raw data goes in and predictions come out. No data leakage.

Scroll to the cross-validation results table

All four models exceeded both success targets. Logistic Regression actually led with AUC 0.698 and recall 0.609, which was a surprise. Random Forest was close behind at 0.696. XGBoost came in third at 0.679, and SVM was last at 0.675 -- but SVM took 45 minutes to train versus 6 seconds for XGBoost. That 45-minute training time means hyperparameter search on SVM would take days, so I excluded it from tuning.

[Scroll to the imbalance comparison chart](#)

This is one of the most important results in the entire notebook. I compared three strategies: `class_weight='balanced'`, SMOTE, and no handling at all. The 'no handling' strategy had a recall of 0.008 -- less than 1%. The model predicted 'not readmitted' for virtually everyone and was right 89% of the time. Completely useless clinically. `class_weight='balanced'` and SMOTE performed comparably, but balanced weights are simpler and don't require the extra memory overhead of SMOTE, so that's what I went with.

Why was LR better than XGBoost at baseline?

XGBoost's default hyperparameters (`learning_rate=0.1`, `max_depth=6`) are rarely optimal for any specific dataset. LR with `class_weight='balanced'` is already well-configured for imbalanced binary classification. This is exactly why Phase 5 exists -- gradient boosting models typically improve the most with tuning.

Phase 5: Hyperparameter Tuning

[Scroll to the tuning strategy markdown](#)

I used a two-stage approach: `RandomizedSearchCV` for broad exploration, then `GridSearchCV` to refine around the best region. The math matters: XGBoost has six tunable parameters. A full grid with four values each is over 4,000 combinations times five folds -- that's 34 hours of training. `RandomizedSearch` with 80 iterations covers the same space in about two hours, and `GridSearch` narrows it down in another hour. Same quality result, 95% less compute.

[Scroll through the three tuning sections \(LR, RF, XGBoost\)](#)

Logistic Regression barely moved -- AUC went from 0.698 to 0.699. It switched from L2 to L1 penalty with `C=0.1`, which provides slightly better feature selection through sparsity, but there just wasn't much room to improve. Total tuning time about nine minutes.

Random Forest improved AUC from 0.696 to 0.701 -- but here's the critical finding: recall collapsed from 0.574 to 0.224. The tuned Random Forest became better at overall discrimination but far more conservative in positive predictions. It misses 78% of actual readmissions. This is a cautionary example of optimizing for AUC alone on imbalanced data.

XGBoost had the biggest improvement: AUC from 0.679 to 0.703. The key parameter changes were learning rate dropping from 0.1 to 0.03 -- slower, more careful boosting -- and depth decreasing from 6 to 4, which reduces overfitting. More iterations at a lower learning rate. And critically, XGBoost was the only model that improved both AUC and recall after tuning. Recall went from 0.509 to 0.638.

[Scroll to the tuning progress log and chart](#)

This chart tells the whole story. XGBoost went from worst baseline to best tuned. The ranking flipped from `LR > RF > XGB` at baseline to `XGB > RF > LR` after tuning. That validates the decision to include XGBoost despite its third-place baseline finish.

Why did RF recall collapse?

Optimizing for AUC means maximizing the model's ability to rank patients correctly across all possible thresholds -- it doesn't guarantee good performance at any specific threshold. The tuned RF (deeper trees, smaller leaves) got better at ranking but shifted its decision boundary toward predicting fewer positives. With 11% prevalence, that's easy to do. This is why we monitor recall alongside AUC -- and why threshold optimization in Phase 6 matters.

Scroll to the final model selection cell

XGBoost was selected as the final model: highest tuned AUC at 0.703, highest tuned recall at 0.638, and the fastest total tuning time at about two minutes. It's also the published benchmark model for this dataset, so the result aligns with the literature. The model was fitted on the full training set and saved. Test set still locked.

Phase 6: Test Set + SHAP

Scroll to 6.1 -- the big reveal

This is the first and only time the test set is used. The big number: test AUC is 0.693, compared to the CV AUC of 0.703. That's a drop of less than 0.01, which means the model generalizes well -- no meaningful overfitting. Both targets are met: AUC 0.693 exceeds the 0.65 target, and recall 0.621 exceeds the 0.50 target.

The confusion matrix shows 613 true positives out of 987 actual readmissions, 374 missed, and 4,577 false positives. Precision is 11.8%, which looks bad until you remember the base rate is 7%. The model intentionally trades precision for recall because a missed readmission costs \$13,000 in penalties while a false positive costs \$500 in intervention.

Scroll to the ROC curves

All three tuned models performed almost identically on the test set -- XGBoost at 0.693, Random Forest at 0.691, LR at 0.691. The tight clustering -- all within 0.003 -- suggests that linear and nonlinear models extract comparable signal from this feature set. The modest AUC ceiling is likely due to missing social determinants, not model limitations.

Scroll to the precision-recall curve

The PR curve is more honest than the ROC for imbalanced data. The average precision is 0.157 - roughly 2.2 times the baseline prevalence of 0.070. That confirms the model adds real signal above random. The steep precision drop-off at high recall shows the fundamental tradeoff: to catch most readmissions, you have to flag a lot of non-readmitted patients too.

Scroll to threshold optimization

The optimal F1 threshold turns out to be 0.50 -- the default. That's unusual for imbalanced data and happens because XGBoost's `scale_pos_weight` already shifts the internal calibration. But F1 isn't necessarily what the hospital should optimize for. Lowering the threshold to 0.46 gets recall above 0.70, catching 711 of 987 readmissions. Lowering it further to 0.30 gets recall above 0.90. The right threshold depends on the hospital's cost structure, which we quantify in the business simulation.

Scroll to SHAP bar plot

SHAP analysis tells us what's driving the model's predictions.

1. The top feature is time_in_hospital -- shorter stays are associated with higher readmission risk, possibly reflecting premature discharge.
2. Second is number_inpatient -- prior hospitalization history.
3. Third is discharge_disposition, with SNF and rehab discharges carrying higher risk than home discharge.
4. Lab procedures
5. Number of diagnoses round out the top five -- both proxies for clinical complexity.

Why is time_in_hospital #1 instead of number_inpatient?

In the EDA, number_inpatient had the strongest univariate correlation with readmission. But SHAP measures importance in the context of all other features simultaneously. Time_in_hospital captures information that overlaps with multiple other features -- complexity, procedure count, medication management -- making it the single most informative signal in the multivariate model.

Scroll to SHAP dot plot

The dot plot shows directional effects. Red dots mean high feature values, blue means low.

- For number_inpatient, you can see red dots spreading right -- more prior visits pushes the prediction toward readmission.
- For discharge_disposition_1 -- which is discharge to home -- red dots pull left, meaning going home is protective.
- The total_visits_prior_year composite reinforces the prior utilization signal.

Scroll to the waterfall plots

These are individual patient explanations -- exactly what a clinician would see at discharge.

- The high-risk patient had a predicted probability of 0.896, driven primarily by six prior inpatient visits contributing +1.48 to the log-odds.
- The low-risk patient at 0.098 lacked that utilization history.
- The borderline patient at 0.500 shows a balanced tug-of-war between risk-increasing and protective features.

This is the interpretability story that makes the model clinically useful.

What happened to has_A1c_measured?

It didn't appear in the top 20 SHAP features. The univariate EDA signal was real -- we confirmed it with a chi-square test -- but it didn't survive multivariate modeling. Its information was likely absorbed by correlated features like diabetesMed and number_diagnoses. This is one of the most honest findings in the project: not every EDA hypothesis holds up when features compete in an ensemble.

Phase 6: Business Impact

[Scroll to the clinical risk factor summary](#)

Before the dollar numbers, I translate the SHAP results into plain English. Ten risk factors, each with a clinical interpretation and hedged 'associated with' language -- these are correlations, not causal claims. Four actionable recommendations: target high-utilizers for transitional care, scrutinize short stays, coordinate by discharge disposition, and flag complex patients. This section is what the CMO reads.

[Scroll to the business impact simulation](#)

The simulation assumes a hospital with 10,000 annual Medicare discharges, \$500 per flagged patient for a transitional care intervention, and \$13,000 in penalty savings per correctly identified readmission. At the recommended threshold of 0.30, the model catches 92% of readmissions and generates \$4.6 million in annual net benefit -- that's \$8.4 million in penalty savings minus \$3.75 million in intervention costs.

The remarkable finding is that net benefit is positive at every threshold tested. Even at the most aggressive setting where you flag nearly everyone, the 26-to-1 ratio of penalty savings to intervention cost means the model pays for itself. The question isn't whether to deploy, it's how aggressively to flag.

[Scroll to the threshold comparison table](#)

At threshold 0.20, you catch 98% of readmissions but flag 9,271 patients -- net benefit \$4.3 million. At 0.30, you catch 92% and flag 7,498 -- net benefit \$4.6 million, the optimal. At 0.50, the default, you catch 62% and flag 3,684 -- net benefit \$3.8 million. The hospital picks based on their intervention capacity. A hospital with limited case managers might operate at 0.50. A hospital with a robust transitional care program can push to 0.30.

Is \$13,000 per readmission realistic?

It's the average Medicare payment per inpatient stay times the average HRRP penalty rate. The actual penalty varies by hospital -- it's calculated as a fraction applied to all Medicare payments, not per readmission. But for modeling purposes, this gives a reasonable per-patient estimate. The key insight is the ratio: even if the true number is \$8,000 instead of \$13,000, the 16-to-1 ratio still makes the model net-positive at every threshold.

Phase 7: Wrap-Up + Q&A

Scroll to the hypothesis revisit table

In Phase 2, I stated four modeling hypotheses. Two were confirmed, one partially confirmed, one not confirmed.

- **CONFIRMED:** discharge disposition matters more than admission type, and tree-based models outperform LR -- though the XGBoost advantage was only 0.003 on the test set.
- **PARTIALLY CONFIRMED:** prior utilization features are prominent but don't solely dominate -- `time_in_hospital` ranked first.
- **NOT CONFIRMED:** `has_A1c_measured` was supposed to be in the top 10 but didn't make the top 20. That's an honest result and shows the value of checking hypotheses against actual model behavior.

Scroll to limitations

Three key limitations to acknowledge. First, the dataset is from 1999-2008. Clinical practices have evolved and ICD-9 has been replaced by ICD-10, so the specific model coefficients wouldn't transfer to a modern hospital without retraining. The HRRP penalty structure is current, but the features would need remapping.

Second, the dataset has no social determinants of health. Housing stability, transportation, caregiver support -- these are known to be among the strongest predictors of readmission and they're completely absent. That's likely the ceiling on AUC. Clinical data alone can't fully predict a phenomenon that's substantially driven by what happens after discharge.

Third, the recommended threshold of 0.30 flags 75% of patients. That requires significant intervention capacity. A resource-constrained hospital might need a higher threshold even if it means catching fewer readmissions.

Scroll to references

Five formal citations: the original Strack paper, the UCI dataset, the CMS HRRP documentation, the Masnoon polypharmacy systematic review, and the Lundberg SHAP paper. Everything in the notebook traces back to published work.

Scroll to the final statistics cell

Final count: 216 cells, 27 figures, five statistical tests, seven engineered features, four models, two-stage tuning, SHAP analysis on 3,000 test patients, and a cost-benefit simulation at seven thresholds. The pipeline is fully reproducible -- Restart and Run All regenerates everything from the raw CSV.

The bottom line: a tuned XGBoost model achieves AUC 0.693 on held-out data, catches 92% of readmissions at the recommended threshold, and generates \$4.6 million in annual net benefit for a 10,000-discharge hospital. The top risk factors are length of stay, prior utilization, and discharge disposition -- each with a specific clinical intervention attached.

Anticipated Questions & Answers

Why was XGBoost worst at baseline but best after tuning?

XGBoost's default hyperparameters (`learning_rate=0.1`, `max_depth=6`) are generic starting points, not optimized for any dataset. The tuned configuration (`learning_rate=0.03`, `max_depth=4`) uses slower, more careful boosting with shallower trees -- less overfitting, better generalization. Gradient boosting models consistently benefit the most from tuning because they have more knobs to turn than LR or RF. The +0.024 AUC improvement was the largest of any model.

Why did Random Forest recall collapse from 0.574 to 0.224?

AUC measures ranking quality across all possible thresholds. The tuned RF got better at ranking patients but shifted its internal decision boundary toward predicting fewer positives. With 11% prevalence, a model can improve its overall discrimination while becoming very conservative at the 0.5 cutoff. This is why we monitor recall alongside AUC, and why the imbalance comparison in Phase 4 was so important -- it proved that without class balancing, recall drops to near zero.

Isn't 11.8% precision terrible?

At 7% base rate, random guessing gives 7% precision. The model is at 11.8% -- nearly 2x better than random. But the real answer is that precision isn't the right metric for this problem. A false positive costs \$500 in unnecessary intervention. A false negative costs \$13,000 in penalty. That 26-to-1 cost ratio means you want to over-flag, not under-flag. At threshold 0.30, precision is only about 8.6% but the net benefit is \$4.6 million because every true positive saves 26x what each false positive costs.

What happened to the HbA1c measurement finding?

The EDA showed a real univariate association -- patients whose HbA1c was measured had lower readmission rates, confirmed by chi-square test. But in the multivariate XGBoost model, that signal was absorbed by correlated features like `diabetesMed` and `number_diagnoses`. `has_A1c_measured` didn't appear in the top 20 SHAP features. This is one of the most instructive findings: univariate significance doesn't guarantee multivariate importance. The ensemble found better proxies for the same underlying clinical behavior.

Could you deploy this model at a real hospital?

Not this exact model -- it was trained on 1999-2008 data with ICD-9 codes. But the pipeline is deployable. You'd retrain on current EHR data with ICD-10 codes, validate on a hold-out period rather than a random split, and package it as a SMART on FHIR app that surfaces patient-level SHAP explanations at the discharge decision point. The architecture -- sklearn Pipeline with ColumnTransformer, XGBoost classifier, SHAP explainer -- transfers directly.

Why use SHAP instead of just feature importance from XGBoost?

XGBoost's built-in importance (gain or frequency) tells you which features the model uses but not the direction or magnitude of their effect. SHAP values are additive -- they decompose each prediction into per-feature contributions that sum to the final output. That gives you directional effects (more inpatient visits pushes toward readmission), patient-level explanations (this patient's risk is driven by factors X and Y), and global importance that's theoretically grounded in cooperative game theory. For clinical stakeholders, the waterfall plots showing why a specific patient was flagged are far more useful than a generic bar chart.

What would you do differently if you started over?

Two things. First, I'd incorporate zip-code-level social determinant indices -- Area Deprivation Index or Social Vulnerability Index -- even though they're not in the dataset. You can merge them by hospital ID or patient zip code if available. The ~0.70 AUC ceiling is almost certainly due to missing SDOH. Second, I'd use probability calibration (Platt scaling or isotonic regression) so the predicted probabilities have clinical meaning -- telling a doctor 'this patient has a 35% chance of readmission' is more useful than 'this patient scored 0.35 on an uncalibrated scale.'

Why is the net benefit positive at every threshold?

Because the cost ratio is so extreme: \$13,000 saved per true positive versus \$500 spent per flagged patient. That's 26 to 1. As long as the model's precision exceeds $1/26 = 3.8\%$, the intervention pays for itself. Our model's precision ranges from 7% at threshold 0.05 to 11.8% at threshold 0.50 -- always above 3.8%. In practice, not all hospitals have this cost structure, and the intervention cost may be higher than \$500 for intensive programs. But the ratio would have to drop below about 4-to-1 before the model stops being net-positive.

Quick Reference: Key Numbers

Metric	Value
Final model	XGBoost (tuned)
Test AUC	0.693 (target: > 0.65)
Test Recall @ 0.50	0.621 (target: > 0.50)
Test Recall @ 0.30	0.919
Precision @ 0.50	0.118
Optimal F1 threshold	0.50 (F1 = 0.199)
Recommended operating threshold	0.30 (\$4.6M net benefit)
SHAP #1 feature	time_in_hospital
SHAP #2 feature	number_inpatient
SHAP #3 feature	discharge_disposition_id
Notebook cells	216 (135 md, 76 code, 27 figures)
Tuning improvement	XGB: +0.024 AUC (worst -> best)
EDA hypotheses confirmed	2 of 4 (1 partial, 1 not)

